

first step is to add the Erlang solutions repository.

```
$ wget https://packages.erlang-solutions.com/erlang-solutions_2.0_all.deb && sudo dpkg -i erlang-solutions_2.0_all.deb
```

After this, *apt-get* can be used to install as shown below:

```
$ sudo apt-get update
$ sudo apt-get install esl-erlang
$ sudo apt-get install elixir
```

The Erlang platform for Elixir is installed using the *apt-get install esl-erlang* command. Detailed installation instructions along various environments such as Raspberry Pi and Docker are given in the official documentation at <https://elixir-lang.org/install.html>.

After the successful installation of Elixir, check the installed version with the following command:

```
$ elixir --version
```

The Elixir interactive mode

The Elixir programming language provides an interactive mode which can be invoked by running the *iex* command at the terminal. A screenshot of the Elixir interactive mode environment and version is shown in Figure 2.

The interactive mode can be used to evaluate any expressions similar to Python.

```
iex(1) > 44 * 2
88
```

```
iex(2)>
```

To come out from the interactive shell, press *Ctrl+C* twice.

Script execution

To execute the Elixir script, type your code in an editor and save it as *.exs*. For example:

```
I0.puts "Welcome to Open Source For You!!"
```

Save the above code as *hello.exs*. Then execute with the Elixir command as shown below:

```
$ elixir hello.exs
```

Language constructs

Learning to code in Elixir is comparatively simpler. If you are already familiar with programming languages such as Python, you will find learning Elixir is easier.

Elixir has support for the following data types: int, float, bool, atom, strings, list and tuples.

```
iex> 8 # integer
iex> 0x1F # integer
iex> 8.0 # float
iex> true # boolean
iex> :atom # atom / symbol
iex> "OSFY" # string
iex> [1, 2, 3] # list
iex> {1, 2, 3} # tuple
```

The Elixir atom is similar to that of some other programming languages'

symbols. An atom in elixir is a constant, whose value is its own name. The atoms come handy when you want to enumerate over distinct values.

Elixir has support for anonymous functions. An anonymous function is one with no name. For example:

```
iex(1)> add = fn a, b -> a + b end
#Function<13.126501267/2 in :erl_eval.expr/5>
```

```
iex(2)> add.(4,4)
8
```

```
iex(3)> is_function(add)
true
```

```
iex(4)>
```

You can observe that in *iex(1)*, we have defined a function with *fn* and *end*. Note that there is no name to the function. It has been stored to a variable *add*. Later, in *iex(2)*, the function has been called using the variable name *add*. Here, *add* is not a function name. You can observe a *'* between *add* and the *'*.

The complete coverage of various language constructs is available in the official documentation.

Elixir processes

The Elixir code runs inside processes, which are isolated from each other. The Elixir's processes are not the same as the operating system processes. The processes inside Elixir are very lightweight in nature, both in terms of memory and CPU utilisation. Due to this, several hundreds of processes can be executed simultaneously. The Elixir processes can communicate among themselves using message passing. These processes facilitate building distributed applications.

A new process can be spawned with *spawn* in Elixir, as shown below:

```
iex(4)> spawn fn -> 1 + 2 end
#PID<0.110.0>
```

```
iex(5)> spawn fn -> 1 + 2 end
```

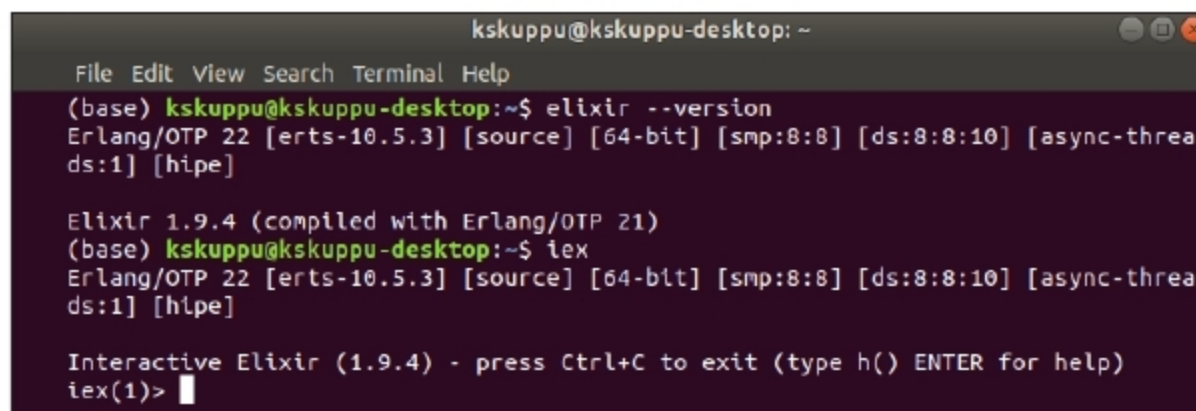


Figure 2: Screenshot of the Elixir interactive mode and version